

MODULE *LamportMutex*

TLA+ specification of *Lamport's* distributed mutual-exclusion algorithm that appeared as an example in L. *Lamport*: Time, Clocks and the Ordering of Events in a Distributed System. *CACM* 21(7):558 – 565, 1978.

EXTENDS *Naturals*, *Sequences*

The parameter N represents the number of processes. The parameter $maxClock$ is used only for model checking in order to keep the state space finite.

CONSTANT N , $maxClock$

ASSUME $NType \triangleq N \in Nat$

ASSUME $maxClockType \triangleq maxClock \in Nat$

$Proc \triangleq 1 \dots N$

$Clock \triangleq Nat \setminus \{0\}$

For model checking, add *ClockConstraint* as a state constraint to ensure a finite state space and override the definition of *Clock* by $1 \dots maxClock + 1$ so that *TLC* can evaluate the definition of *Message*.

VARIABLES

<i>clock</i> ,	local clock of each process
<i>req</i> ,	requests received from processes (clock transmitted with request)
<i>ack</i> ,	acknowledgements received from processes
<i>network</i> ,	messages sent but not yet received
<i>crit</i>	set of processes in critical section

Messages used in the algorithm.

$ReqMessage(c) \triangleq [type \mapsto "req", clock \mapsto c]$

$AckMessage \triangleq [type \mapsto "ack", clock \mapsto 0]$

$RelMessage \triangleq [type \mapsto "rel", clock \mapsto 0]$

$Message \triangleq \{AckMessage, RelMessage\} \cup \{ReqMessage(c) : c \in Clock\}$

The type correctness predicate.

$TypeOK \triangleq$

$clock[p]$ is the local clock of process p

$\wedge clock \in [Proc \rightarrow Clock]$

$req[p][q]$ stores the clock associated with request from q received by p , 0 if none

$\wedge req \in [Proc \rightarrow [Proc \rightarrow Nat]]$

$ack[p]$ stores the processes that have *ack*'ed p 's request

$\wedge ack \in [Proc \rightarrow SUBSET Proc]$

$network[p][q]$: queue of messages from p to q – pairwise *FIFO* communication

$\wedge network \in [Proc \rightarrow [Proc \rightarrow Seq(Message)]]$

set of processes in critical section: should be empty or singleton

$\wedge crit \in SUBSET Proc$

The initial state predicate.

$$\begin{aligned} & \wedge network' = [network \text{ EXCEPT } ![q][p] = Tail(@)] \\ & \wedge \text{UNCHANGED } \langle clock, req, crit \rangle \end{aligned}$$

Process p enters the critical section.

$$\begin{aligned} Enter(p) & \triangleq \\ & \wedge ack[p] = Proc \\ & \wedge \forall q \in Proc \setminus \{p\} : beats(p, q) \\ & \wedge crit' = crit \cup \{p\} \\ & \wedge \text{UNCHANGED } \langle clock, req, ack, network \rangle \end{aligned}$$

Process p exits the critical section and notifies other processes.

$$\begin{aligned} Exit(p) & \triangleq \\ & \wedge p \in crit \\ & \wedge crit' = crit \setminus \{p\} \\ & \wedge network' = [network \text{ EXCEPT } ![p] = Broadcast(p, RelMessage)] \\ & \wedge req' = [req \text{ EXCEPT } ![p][p] = 0] \\ & \wedge ack' = [ack \text{ EXCEPT } ![p] = \{\}] \\ & \wedge \text{UNCHANGED } clock \end{aligned}$$

Process p receives a release notification from q .

$$\begin{aligned} ReceiveRelease(p, q) & \triangleq \\ & \wedge network[q][p] \neq \langle \rangle \\ & \wedge \text{LET } m \triangleq Head(network[q][p]) \\ & \quad \text{IN } \wedge m.type = \text{"rel"} \\ & \quad \wedge req' = [req \text{ EXCEPT } ![p][q] = 0] \\ & \quad \wedge network' = [network \text{ EXCEPT } ![q][p] = Tail(@)] \\ & \quad \wedge \text{UNCHANGED } \langle clock, ack, crit \rangle \end{aligned}$$

Next-state relation.

$$\begin{aligned} Next & \triangleq \\ & \vee \exists p \in Proc : Request(p) \vee Enter(p) \vee Exit(p) \\ & \vee \exists p \in Proc : \exists q \in Proc \setminus \{p\} : \\ & \quad ReceiveRequest(p, q) \vee ReceiveAck(p, q) \vee ReceiveRelease(p, q) \\ vars & \triangleq \langle req, network, clock, ack, crit \rangle \\ Spec & \triangleq Init \wedge \Box [Next]_{vars} \end{aligned}$$

A state constraint that is useful for validating the specification using finite-state model checking.

$$ClockConstraint \triangleq \forall p \in Proc : clock[p] \leq maxClock$$

No channel ever contains more than three messages. In fact, no channel ever contains more than one message of the same type, as proved below.

$$BoundedNetwork \triangleq \forall p, q \in Proc : Len(network[p][q]) \leq 3$$

The main safety property of mutual exclusion.

$$Mutex \triangleq \forall p, q \in crit : p \neq q$$